

Roadmap

学习路线

▼ 提示

本文章正在编辑讨论中，欢迎补充更进一步的学习路线或在评论区提出你的想法！

本文将会介绍算法竞赛的学习路线。

该学习路线既是新手学习算法竞赛知识的指南，也是一份复习清单。

1 C++ 语言基础

先从 C++ 语法学起，一步一步来。

1.1 Hello, World!

以一句 `Hello, World!`，开始算法竞赛之旅吧！

同时了解一下 C++ 的源程序的大致框架是什么样子的。

- [Hello, World!](#)
- [C++ 语法基础](#)

1.2 变量与运算

计算机出现的最初目的就是计算。因此我们先学习如何完成一些简单的运算任务吧。

- [变量](#)
- [运算](#)

1.3 流程控制

1.3.1 分支结构

有的时候，我们需要在不同的条件下，选择执行不同的语句，这时候我们就需要借助分支语句。

- [分支](#)

分支语句包括下面几种：

- if 语句
- if-else 语句
- if-elif-else 语句
- switch 语句

1.3.2 循环结构

将若干条语句重复执行多次，就需要用到循环语句。

- [循环](#)

循环语句包括下面几种：

- [for 语句](#)
- [while 语句](#)
- [do-while 语句](#)

1.4 数组与结构体

数组用于存储大量相同类型的数据。而结构体则可以将若干变量捆绑起来。

- [数组](#)
- [结构体](#)

1.5 函数与递归

使用函数来让程序变得模块化，降低实现成本。

递归则是新手入门的一道坎，「自己调用自己」听起来并不是那么容易理解，不过仔细深究根本，就会发现「自己调用自己」和「自己调用别人」并没有本质差别。

- [函数](#)
- [递归 & 分治](#)

2 CSP-J 入门级

2.1 枚举与模拟

从现在开始，你已经会使用 C++ 语言完成一些简单的任务了，但是这远远不够。

为了对一些简单的题目，你需要学会通过枚举或模拟脑海中的逻辑，来实现代码。这看起来并不是很高效，但有的时候很管用。

- [枚举](#)
- [模拟](#)

2.2 递归与分治

递归是指函数定义中不断调用自己的方法；而分治则是不断将这一个问题分解为若干子问题，求解后合并的操作。

- [递归 & 分治](#)

2.3 字符串

在做信息学题目时，经常会碰到的一个数据类型就是字符串，你需要学习一些用于操作字符串的 STL 函数。当然，模拟也是解决字符串问题的好方法。

- [字符串基础](#)
- [STL 函数](#)

2.4 排序

当你获得了一组数据时，如何将他们从无序变成有序也是个很重要的问题。在你没有思路的时候，不妨考虑一下将数组排个序吧。这也是接下来的很多算法的基础。

排序的方法有点多，但理解后记住它们并不难。

- [排序简介](#)
- [选择排序](#)
- [冒泡排序](#)

- [插入排序](#)
- [计数排序](#)
- [基数排序](#)
- [快速排序](#)
- [归并排序](#)
- [堆排序](#)
- [桶排序](#)
- [排序相关 STL](#)

NOI 大纲中入门级只要求学习选择、冒泡、插入排序，共三个排序算法，但是其余的难度也并不大，且初赛中可能涉及，故一并列出。

2.5 二分与倍增

二分查找，本质上是运用分治的思想，不断减少查找范围的大小，直至找到答案。但是需要注意，这个查找方式必须应用在有序的数据结构中。

- [二分](#)

而倍增则不同，它是不断翻倍，以把线性范畴内的处理转化为对数级，大大优化时间复杂度。（这个知识点需要一点数学基础，暂时跳过也问题不大）

- [倍增](#)

2.6 搜索

在入门组，搜索的题目常常会在迷宫类题目中出现，一般会有地图类的数据；此外，搜索也十分常用于高效地枚举构造合法解的情况，亦可用于骗分。

2.6.1 深度优先搜索（DFS）

深度优先搜索指利用递归函数方便地实现暴力枚举的算法，与图论中的 DFS 算法有一定相似之处，但并不完全相同。

- [DFS（搜索）](#)

2.6.2 广度优先搜索（BFS）

将每一个状态设计为图中的一个点，可以展开地毯式搜索。

- [BFS（搜索）](#)

2.6.3 搜索优化

很多题目都可以用 DFS 来解决，而这个算法的复杂度显然是无法通过的。因此，需要一些优化使它跑得更快。这样的优化能够减少不可能成功的尝试，称为「剪枝」。BFS 相关的优化就要更加灵活了，但是基本思路和这里是一样的。

- [DFS 剪枝优化](#)

2.7 数据结构入门

2.7.1 线性数据结构

数组，链表，队列，栈，都是线性结构。巧用这些结构可以做出不少方便的事情。

- [栈](#)
- [队列](#)
- [链表](#)

2.7.2 复杂数据结构

- [树及二叉树](#)
- [图的概念](#)
- [图的存储](#)

2.8 动态规划入门

动态规划 (Dynamic Programming, DP) 是一种通过把原问题分解为相对简单的子问题的方式求解复杂问题的方法。

由于动态规划并不是某种具体的算法，而是一种解决特定问题的方法，因此它会出现于各式各样的数据结构中，与之相关的题目种类也更为繁杂。

- [动态规划简介](#)

2.8.1 背包问题

即给出一个有限容量的背包，选择放入若干有容量和价值的物品，求解如何放置能使得价值总和最大。这是阻挡很多 Oler 的第一道坎，从这里开始，算法就有些难以理解。

- [背包 DP](#)

2.8.2 线性动态规划

在动态规划中，最难的部分之一就是设计状态，需要用到构造相关技巧。当你写出了状态和状态转移方程之后，完成一道动态规划的题目就不难了。

- [构造](#)
- [动态规划基础](#)

记忆化搜索是一种通过记录已经遍历过的状态的信息，从而避免对同一状态重复遍历的搜索实现方式。有的题目也可以使用记忆化搜索来降低思维难度。

因为记忆化搜索确保了每个状态只访问一次，它也是一种常见的动态规划实现方式。

- [记忆化搜索](#)

2.8.3 复杂动态规划

区间类动态规划是线性动态规划的扩展，它在分阶段地划分问题时，与阶段中元素出现的顺序和由前一阶段的哪些元素合并而来有很大的关系。

- [区间 DP](#)

2.9 数学

2.9.1 高精度算法

就算是 long long (或 int64) 还不够怎么办？用高精度算法。本质上就是模拟了四则运算。

- [高精度计算](#)

2.9.2 进制转换

在计算机中，除了二进制，比较常用的还有八进制和十六进制。有的时候学会运用正确的进制对解题也有很大帮助。

- [进位制](#)

2.9.3 位运算

位运算就是基于整数的二进制表示进行的运算。由于计算机内部就是以二进制来存储数据，位运算是相当快的。

基本的位运算共 6 种，分别为按位与、按位或、按位异或、按位取反、左移和右移。

- [位运算](#)

2.9.4 数论

- [数论基础](#)
- [素数](#)
- [筛法](#)
- [最大公因数](#)
- [欧拉函数](#)
- [分解质因数](#)

2.9.5 组合计数

- [排列组合](#)
- [抽屉原理](#)
- [容斥原理](#)

至此，你就学习完了入门组范畴内的所有算法，但是想要掌握它们，你需要继续进行足够数量的刷题，以巩固你所学到的知识点。

本页面最近更新：2023/9/28 07:21:54, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [CBW2007](#), [CCXXI](#), [CLCK0622](#), [Enter-tainer](#), [Great-designer](#), [lr1d](#), [Konano](#), [ksyx](#), [StudyingFather](#), [Tiphereth-A](#), [yuhuoji](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用